

Hardening compute: AMIs, roles, IMDSv2 and Session Manager

The image you launch, the identity it carries, and how a human reaches it.

You cannot harden a running fleet. You harden the **image**, and then replace the fleet.

Golden AMIs and EC2 Image Builder

- ◆ A **pipeline** runs a **recipe** – a base image plus **components** – on a schedule or when the base image updates.
- ◆ AWS supplies hardening components, including **CIS benchmark** components, so you are not writing hardening scripts yourself.
- ◆ It **tests** the image before distribution, and **distributes** it to the Regions and accounts you nominate.
- ◆ Bake in the **SSM Agent**, the **CloudWatch agent** and **GuardDuty Runtime Monitoring**, as module 8 argued.
- ◆ Image Builder can also produce **container images**, not only AMIs.

ON THE EXAM

An image nobody can explain the provenance of is a supply chain risk.

Patch Manager, for what you cannot replace

- ◆ **Patch baselines** define which patches are approved, with auto-approval delays so a patch soaks before it lands everywhere.
- ◆ **Patch groups** are just a tag, letting different baselines apply to production and non-production.
- ◆ **Maintenance windows** control when patching runs. **Scan** reports compliance without installing anything.
- ◆ Compliance data flows to Systems Manager and can be surfaced in Security Hub and Config.
- ◆ Patch Manager **needs the SSM Agent and an instance profile**. An unmanaged instance is invisible to it, and reports nothing.

ON THE EXAM

Inspector finds the CVE. Patch Manager installs the fix. Neither does the other's job, and the exam offers them as alternatives.

IMDSv2, and why it defeats SSRF

- ◆ **IMDSv1** answers a plain HTTP GET. Any process, or any SSRF flaw, can read the instance role's credentials.
- ◆ **IMDSv2** requires a **PUT** to obtain a session token first, which a simple SSRF cannot issue.
- ◆ It also **rejects requests carrying X-Forwarded-For**, so a request that passed through a proxy is refused.
- ◆ Session tokens last from one second to six hours. Set **HttpTokens=required** to refuse IMDSv1 entirely.
- ◆ You can switch a **running** instance to required without stopping it.

ON THE EXAM

Security Hub control EC2.8 and CIS control 5.6 both check this. It is the most-audited single EC2 setting there is.

Enforcing IMDSv2 across an estate

- ◆ **Account and Region defaults** – `modify-instance-metadata-defaults` sets `HttpTokens`, hop limit and endpoint for new instances.
- ◆ The shipped value is **no-preference**. Nothing is enforced until you set it.
- ◆ An **SCP denying `ec2:RunInstances`** when **`ec2:MetadataHttpTokens`** is not required blocks non-compliant launches.
- ◆ Recent AWS and Marketplace AMIs carry **`ImdsSupport v2.0`**, making required the default.
- ◆ Find stragglers with **Security Hub EC2.8**; fix in place with `modify-instance-metadata-options`.

ON THE EXAM

Defaults cover normal launches. The SCP covers deliberate ones.

The hop limit, and the container problem

- ◆ **HttpPutResponseHopLimit defaults to 1**, range 1 to 64 – how far the token response may travel.
- ◆ **A hop limit of 1 stops containers getting a token**; a container sits behind an extra hop.
- ◆ Raising it to 2 makes containers work – **and lets any container read the instance role's credentials**.
- ◆ Better: **leave it at 1** and give containers their own identity – ECS task roles, or EKS Pod Identity.
- ◆ Raising the limit trades blast radius for convenience. Know that you are doing it.

ON THE EXAM

The fix that makes containers work also gives every container the host's credentials.

Which role does what

ROLE	USED BY	FOR
EC2 instance profile	Code on the instance, via IMDS	Whatever your application calls
ECS task role	The containers in a task	What your application code needs
ECS task execution role	The ECS agent, not your code	Pulling the image, writing logs
Lambda execution role	The function	Its own AWS calls, plus CloudWatch Logs
EKS Pod Identity / IRSA	A Kubernetes service account	Per-pod AWS permissions

ON THE EXAM

Task role and task execution role is the pair most often confused. One is your code; the other is the platform acting on your behalf.

Session Manager

- ◆ **No inbound ports, no bastion, no SSH keys.** The agent connects outbound.
- ◆ Works in private subnets with **no internet**, via VPC endpoints for ssm, ssmmessages and ec2messages.
- ◆ **Every session is logged** – output to S3 or CloudWatch Logs, start and end in CloudTrail.
- ◆ Access is **IAM**, so grant per instance by tag and revoke by policy.
- ◆ Also **port forwarding** and **RunAs**, so engineers connect as a named user, not root.

ON THE EXAM

No key to leak, no port to scan, and no session without a record.

Session Manager, or EC2 Instance Connect?

SESSION MANAGER

Managed shell, fully logged

- No inbound ports at all
- Session output logged to S3 or CloudWatch
- Works on EC2, on-premises and Fargate
- Port forwarding and RunAs

EC2 INSTANCE CONNECT

Short-lived SSH keys

- Pushes a one-off public key, valid 60 seconds
- Still SSH, so still port 22
- Endpoint variant avoids a public IP
- You keep your normal SSH tooling

THE TIEBREAKER

Both remove long-lived keys. Only Session Manager removes the port and records what was typed.

Containers and functions

- ◆ **Scan images with Inspector enhanced scanning**, which rescans as new CVEs appear.
- ◆ Run as a **non-root user** with a **read-only root filesystem**; drop unneeded capabilities.
- ◆ Give each task its own **task role**. A shared role is the module 8 blast radius problem.
- ◆ **Secrets injected at runtime** from Secrets Manager or Parameter Store – never in the image.
- ◆ **GuardDuty Runtime Monitoring** sees a process start inside the container.

ON THE EXAM

A container image is an artefact with a supply chain. Provenance matters.

An audit of a company's 900 EC2 instances finds 210 still accepting IMDSv1. The security team remediates all 210 in place. Six weeks later a fresh audit finds 47 new instances accepting IMDSv1, launched by three different teams from their own launch templates.

The team wants every future instance to require IMDSv2, with non-compliant launches prevented rather than corrected afterwards.

What should the team implement?

- ☐ A Re-run the remediation script weekly against every instance in every account
- ☐ B An AWS Config rule for IMDSv2 with automatic SSM Automation remediation
- ☐ C Account and Region level metadata defaults, plus an SCP denying RunInstances unless ec2:MetadataHttpTokens is required
- ☐ D Update every team's launch template to set HttpTokens to required

◆ PAUSE AND DECIDE

What should the team implement?

- A

 Re-run remediation weekly
The approach that already failed once. It corrects rather than prevents, and leaves up to a week where instances run with IMDSv1 enabled.
- C

 Metadata defaults plus an SCP on ec2:MetadataHttpTokens
Correct. Defaults cover ordinary launches, the SCP refuses a deliberate non-compliant launch, and neither depends on teams changing anything.
- B

 Config rule with automatic remediation
Better than a script and still corrective. There is always a window between launch and remediation, and the requirement says prevented.
- D

 Update every team's launch template
Works for those templates until someone creates a new one or launches without a template. It also needs three teams to act and keep acting.

THE SIGNAL

'Prevented rather than corrected afterwards' is the phrase. Everything corrective is eliminated before you read further.

A company enforces IMDSv2 with a hop limit of 1 across its EC2 fleet. Immediately, containerised applications running on ECS with the EC2 launch type begin failing with credential errors, because they can no longer obtain a metadata token.

A platform engineer proposes raising the hop limit to 2 across the fleet to restore service. The security architect is asked to review this before it is applied.

What should the architect recommend?

- ☐ A Approve it. A hop limit of 2 is the documented setting for containerised workloads
- ☐ B Revert to IMDSv1 for the affected hosts until the applications are redesigned
- ☐ C Approve it, but only on hosts running containers, and add a Config rule to detect drift
- ☐ D Keep the hop limit at 1 and give each task its own ECS task role

◆ PAUSE AND DECIDE

What should the architect recommend?

- A

 Approve; 2 is documented for containers
True and incomplete. It works, and it lets every container on the host read the instance role's credentials - which is what the applications are doing wrong.
- B

 Revert to IMDSv1
Trades a container inconvenience for the SSRF exposure the whole exercise was meant to close. Strictly the worst option here.
- C

 Approve on container hosts only, with drift detection
More careful than A and the same underlying exposure on exactly the hosts that matter. Scoping a weakness is not removing it.
- D

 Hop limit 1, and a task role per task
Correct. Containers get their own scoped identity, the instance role stays unreachable from inside them, and the SSRF protection is preserved.

THE SIGNAL

The containers were using the host's credentials all along. The failure did not create that problem; it revealed it.

A defence contractor runs application servers in private subnets with no internet gateway and no NAT. Engineers currently reach them through a bastion host in a public subnet using shared SSH keys.

An auditor requires that no inbound ports are open on the servers, that no long-lived credentials are used for access, and that a record of the commands run in each session is retained for two years.

What should the contractor implement?

- ☐ A Session Manager with VPC endpoints, and session output logged to an S3 bucket
- ☐ B EC2 Instance Connect Endpoint, replacing the bastion host
- ☐ C Client VPN with certificate authentication, and SSH from the engineer's laptop
- ☐ D Keep the bastion but rotate the shared SSH key weekly with Secrets Manager

◆ PAUSE AND DECIDE

What should the contractor implement?

- A

 Session Manager with VPC endpoints and S3 session logging
Correct. No inbound port at all, IAM instead of keys, and session output written to S3 – which is the only option satisfying all three requirements.
- B

 EC2 Instance Connect Endpoint
Strong on the first two requirements and it is still SSH, so port 22 must be reachable, and it records the connection rather than what was typed.
- C

 Client VPN plus SSH
Puts engineers on the network and still needs port 22 open. It also produces no record of session content.
- D

 Bastion with weekly key rotation
A shared key rotated weekly is still shared and still long-lived by the auditor's standard, and it keeps both the bastion and the open port.

THE SIGNAL

Three requirements, and the third – a record of the commands run – is the one only Session Manager meets.

Four teams each build their own AMIs by launching a base image, running ad hoc scripts, and taking a snapshot. The images differ in patch level, agent versions and OS configuration. Nobody can say what is in the oldest one.

The security team wants one hardened, tested, CIS-benchmarked image, rebuilt automatically when the base image is updated, and distributed to every account and Region that needs it.

What should the security team implement?

- ☐ A A Config rule requiring instances to be launched only from approved AMI IDs
- ☐ B An EC2 Image Builder pipeline with hardening components, tests and distribution settings
- ☐ C Systems Manager Patch Manager on a weekly maintenance window across all instances
- ☐ D Amazon Inspector CIS benchmark assessments on every running instance

◆ PAUSE AND DECIDE

What should the security team implement?

- A

 Config rule for approved AMI IDs
A useful companion control once you have an approved image. On its own it enforces use of images that do not exist yet.
- B

 EC2 Image Builder pipeline
Correct. Recipe plus CIS hardening components, automated rebuild on base image update, tests before release, and distribution to accounts and Regions.
- C

 Patch Manager on a weekly window
Patches running instances and leaves four divergent images being launched every day. It treats the fleet rather than the source.
- D

 Inspector CIS assessments on running instances
Tells you how far each instance deviates from the benchmark. Reporting drift is not producing a hardened image.

THE SIGNAL

Every requirement in the scenario – hardened, tested, rebuilt on update, distributed – is a named Image Builder feature.

Five things to carry forward

- ◆ **Harden the image, then replace the fleet.** Fixing running instances is treating the symptom.
- ◆ **Metadata defaults plus an SCP on ec2:MetadataHttpTokens.** Defaults for normal launches, SCP for deliberate ones.
- ◆ **Hop limit 1 blocks containers – and that is the point.** Give them task roles rather than raising it.
- ◆ **Task role is your code. Task execution role is the ECS agent.** Pulling images and writing logs is the platform's job.
- ◆ **Session Manager logs what was typed.** Instance Connect logs that you connected.

ON THE EXAM

Next: finding the holes – Inspector, runtime monitoring, CodeGuru Security and Amazon Q Developer.